

aelion

Spec IT · Séance 11

Fil rouge InduSense 4.0

POC ML & métriques métier

Accuracy vs Recall

1

Choisir et justifier 3 algorithmes baseline pour un problème de classification tabulaire

2

Calculer et interpréter accuracy, precision, recall, F1-score et matrice de confusion

3

Relier les métriques au coût métier (FN = panne manquée, FP = fausse alarme)

4

Structurer un plan d'expérience et une note d'arbitrage

Product Owner

« On a nettoyé les données ! On peut enfin prédire les pannes.

Si on obtient 98 % d'accuracy, c'est gagné, non ? »

Developer

« 98 % sur un dataset déséquilibré... Un modèle qui prédit TOUJOURS “pas de panne” atteint déjà 98 %.

Il faut choisir les bonnes métriques : Recall d'abord ! »

⚠ L'accuracy est trompeuse sur les datasets déséquilibrés — c'est le piège #1 du ML

01

Choisir un modèle baseline

LR · Random Forest · XGBoost

02

Métriques & coût métier

Accuracy · Recall · F1 · FN/FP

03

Plan d'expérience

Hypothèse · Mesure · Décision

04

TP InduSense

3 modèles · Matrices · Note d'arbitrage

01

Choisir un modèle baseline



Logistic Regression · Random Forest · XGBoost

Classification binaire

panne / pas de panne

Sortie : probabilité ou classe

Données tabulaires

Capteurs température, pression

*Historique incidents · Pas besoin de
Deep Learning*

Apprentissage supervisé

On entraîne sur le passé pour prédire
le futur

Label connu : panne réelle

InduSense : prédire si une machine va tomber en panne dans les 24 h à partir des capteurs de température et pression

La règle des 3 baselines

Toujours commencer par au minimum 3 algorithmes avant de complexifier

aelion

Régression Linéaire

Référence linéaire

Rapide · Interprétable

Baseline incontournable

Random Forest

Robuste · Peu de tuning

Gestion outliers native

Feature importance intégrée

XGBoost

Souvent SOTA tabular

Gestion déséquilibre

Plus de paramètres

→ Commencer par RF, comparer XGBoost, LR comme référence

Principe

Ensemble de N arbres de décision entraînés sur des sous-ensembles aléatoires de données et de features.

Prédiction = vote majoritaire des N arbres.

- Robuste au bruit et aux outliers
- Peu sensible aux hyperparamètres
- Feature importance intégrée
- Faible risque d'overfitting

Hyperparamètres clés : `n_estimators` · `max_depth` · `min_samples_leaf`

Python · `scikit-learn`

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(
    n_estimators=100,
    max_depth=10,
    min_samples_leaf=5,
    random_state=42
)
rf.fit(X_train, y_train)
y_pred = rf.predict(X_test)

# Feature importance
import matplotlib.pyplot as plt
importances = rf.feature_importances_
plt.barh(feature_names, importances)
```

XGBoost — boosting séquentiel

aelion.fr

Principe : chaque arbre corrige les erreurs du précédent (apprentissage séquentiel). Plus puissant que RF sur tabular — mais nécessite plus de tuning.

Random Forest

Parallèle · Vote majoritaire

Peu de tuning

Robuste par défaut

Choisir XGBoost quand RF plafonne et que le temps de tuning est disponible

Python · XGBoost

```
import xgboost as xgb

xgb_model = xgb.XGBClassifier(
    n_estimators=100,
    learning_rate=0.1,
    max_depth=6,
    scale_pos_weight=10, # gestion déséquilibre
    random_state=42,
    eval_metric='logloss'
)
xgb_model.fit(X_train, y_train,
              eval_set=[(X_val, y_val)],
              verbose=False)
```

`scale_pos_weight = nb_négatifs / nb_positifs` — crucial pour les pannes rares

Critères de choix — LR vs RF vs XGBoost

aelion

Critère	LR	Random Forest	XGBoost
Complexité	Faible	Moyenne	Haute
Interprétabilité	Haute	Moyenne	Faible
Perf. tabular	Moyenne	Bonne	Très bonne
Coût calcul	Faible	Moyen	Moyen–Haut
Robustesse outliers	Faible	Bonne	Bonne
Gestion déséquilibre	Manuelle	class_weight	scale_pos_weight

→ InduSense : commencer par Random Forest (robuste, peu de tuning), puis challenger avec XGBoost

02

Métriques & coût métier



Accuracy · Recall · F1 · FN / FP

98 %

d'accuracy

Dataset InduSense

10 000 échantillons

dont 200 pannes (2 %)

Un modèle naïf qui prédit TOUJOURS « pas de panne » :

Accuracy = 98 % ✓

Pannes détectées = 0 ✗

Ne jamais juger un modèle sur le seul critère de l'accuracy quand le dataset est déséquilibré — toujours regarder Recall, Precision, F1.

Matrice de confusion

	Prédit : Panne	Prédit : OK
Réel : Panne	TP Vrai Positif Panne prédite = panne réelle ✓	FN Faux Négatif Panne manquée → DANGER ✗
Réel : OK	FP Faux Positif Fausse alarme → coût inutile	TN Vrai Négatif OK prédit = OK réel ✓

FN = priorité !

Panne manquée = coût élevé

Precision — Parmi les alarmes, combien sont réelles ?

aelion.fr

Formule

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

Parmi toutes les alarmes levées par le modèle, combien correspondent à de vraies pannes ?

aelion



10 alarmes levées



8 vraies pannes

Precision = 8/10 = 80 % → 2 fausses alarmes / 10 alarmes levées

Precision haute = peu de fausses alarmes → moins de maintenances inutiles

Recall — Parmi les pannes réelles, combien détectées ?

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

Parmi toutes les vraies pannes qui ont eu lieu, combien le modèle en a-t-il détectées ?



10 pannes réelles



9 détectées

Recall = 9/10 = 90 % → 1 panne manquée sur 10 = 1 arrêt non planifié

Moyenne harmonique de Precision et Recall

$$F1 = 2 \times (Precision \times Recall) / (Precision + Recall)$$

La moyenne harmonique est plus sévère que la moyenne arithmétique :
un F1 élevé exige que Precision ET Recall soient tous les deux bons.

aelion

Precision = 90 %
Recall = 90 %

F1 = 90 %

Precision = 99 %
Recall = 50 %

F1 = 66 %

Precision = 50 %
Recall = 50 %

F1 = 50 %

Pour InduSense : Recall > F1 > Precision

FN = panne réelle non détectée par le modèle

Arrêt machine non planifié

4 à 8 h d'arrêt de production

Dommages matériels

Remplacement pièces ou machine

Pénalités client

Délais de livraison non respectés

Coût humain

Stress équipe, heures supplémentaires

Coût estimé : 5 000 € à 50 000 € par incident (selon l'industrie et la durée d'arrêt)

Coût métier — Faux Positif (fausse alarme)

FP = alarme levée mais pas de panne réelle

Intervention maintenance inutile

2 à 4 h de travail technicien

Arrêt machine préventif injustifié

1 à 2 h de perte production

Perte de confiance

Si trop de fausses alarmes, les équipes ignorent le système

Coût direct

Main d'œuvre + pièces inutilisées

Coût estimé : **200 € à 500 € par fausse alarme** · **FN coûte 10 à 100× plus cher que FP** → maximiser le Recall

Quelle métrique choisir ?

aelion

Maintenance prédictive (pannes rares)

Recall

Détection de fraude

Recall + Precision

Diagnostic médical

Recall (FN dangereux)

Filtre spam

Precision (FP agaçants)

Dataset équilibré

F1 ou Accuracy

InduSense → Priorité

Recall panne !

03

Code & visualisation



sklearn · heatmap · interprétation pour le PO

Code sklearn — Calculer et afficher les métriques

Python · scikit-learn + matplotlib

```
from sklearn.metrics import (
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay,
    recall_score, f1_score
)
import matplotlib.pyplot as plt

y_pred = rf.predict(X_test)

# Rapport complet
print(classification_report(y_test, y_pred,
    target_names=['OK', 'Panne']))

# Recall et F1 isolés
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
print(f"Recall : {recall:.3f} | F1 : {f1:.3f}")

# Matrice de confusion
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=['OK', 'Panne']
)
disp.plot(cmap='Blues', colorbar=False)
plt.title('Matrice de confusion - Random Forest')
plt.savefig('confusion_matrix_rf.png', dpi=150)
```

Sortie console

	precision	recall	f1-score
OK	0.99	0.98	0.99
Panne	0.72	0.85	0.78
accuracy		0.98	
macro avg	0.86	0.92	0.89

Recall Panne = 0.85

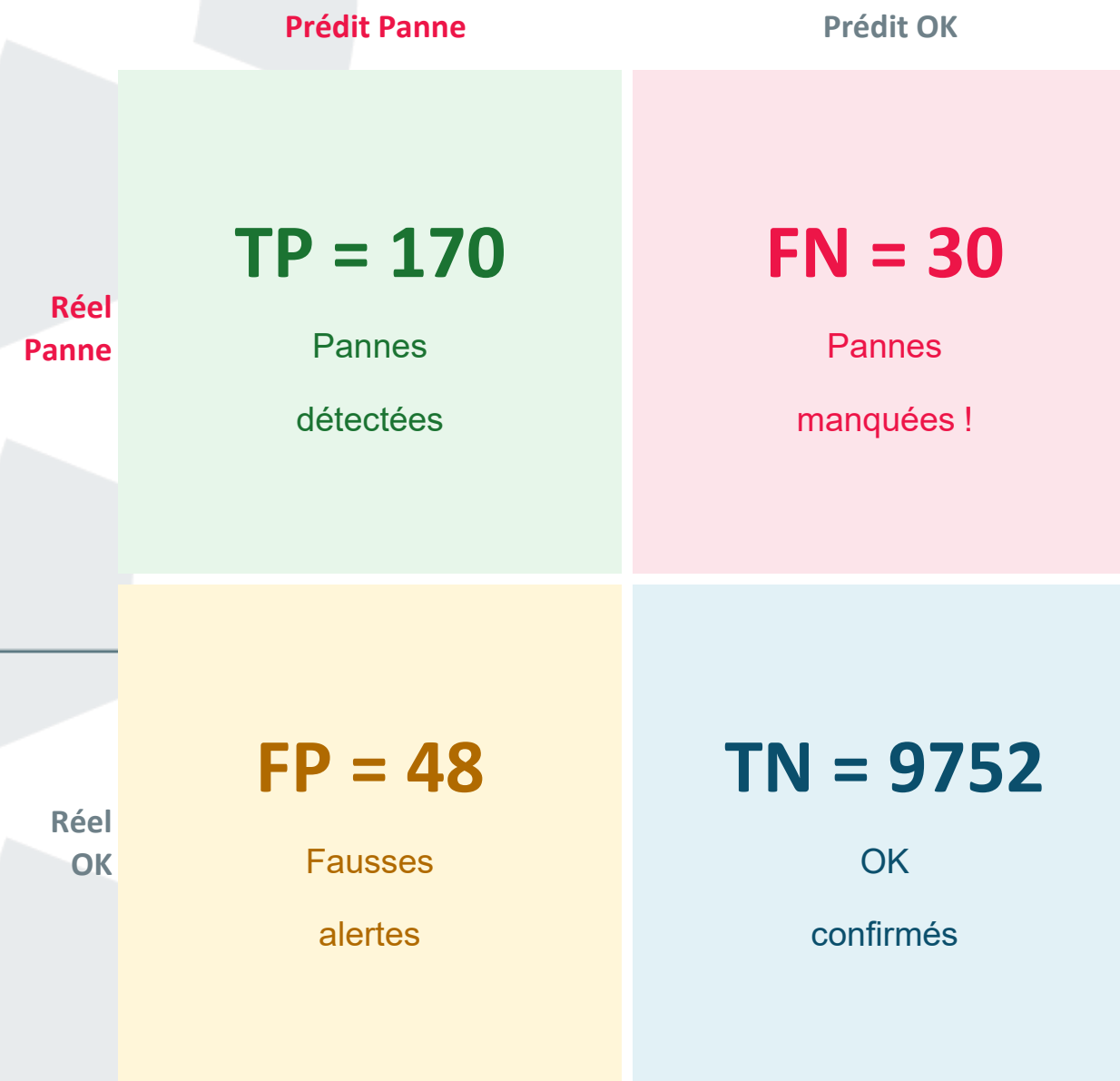
→ 15 % des pannes non détectées

Accuracy = 98 % (trompeuse !)

Recall = 85 % (ce qui compte vraiment)

Visualisation — Heatmap de la matrice de confusion

aelion.fr



Lecture de la heatmap

Recall = $170 / (170+30) = 85 \%$

→ 30 pannes manquées !

Precision = $170 / (170+48) = 78 \%$

→ 48 fausses alarmes

Accuracy = $9\,922 / 10\,000 = 99 \%$

→ Très trompeuse !

Langage technique	Langage métier (pour le PO)
Recall = 85 %	Sur 100 pannes, on en détecte 85. 15 passent inaperçues.
FN = 30	30 pannes manquées sur 200 = 30 arrêts non planifiés potentiels
FP = 48	48 fausses alarmes = 48 interventions inutiles
F1 = 0.78	Score global de fiabilité : 78 % — acceptable pour un POC
AUC = 0.92	Le modèle distingue bien les machines saines des malades (92 %)

—• Toujours raisonner **en coût métier**

Accuracy = trompeuse · **Recall** = détecter les pannes · **Precision** = éviter les fausses alarmes · **F1** = équilibre

04

Plan d'expérience



Hypothèse · Mesure · Décision · reproductibilité



L'objectif n'est pas de trouver le modèle parfait, mais de prendre une décision informée et documentée.

Quelles variantes comparer et pourquoi ?

Définir a priori — avant de lancer les expériences

Modèles à tester

RF · XGBoost · Logistic Regression

(+ 1 variante : RF avec class_weight='balanced')

Métrique principale

Recall panne — objectif : Recall ≥ 0.80

(= ne rater que 20 % des pannes au maximum)

Métriques secondaires

F1-score · Temps d'entraînement

Interprétabilité · Taille du modèle

Contraintes

Temps d'inférence < 100 ms

Interprétable pour le PO · Reproductible (random_state=42)

Modèle	Hyperparams clés	Métrique cible	Hypothèse	Résultat
Random Forest	n_est=100 depth=10	Recall ≥ 0.80	Baseline robuste	?
XGBoost	lr=0.1 n_est=100 scale_pw=10	Recall ≥ 0.85	Meilleur tabular	?
Logistic Reg.	C=1.0 class_w=bal.	Recall ≥ 0.70	Référence linéaire	?
RF balanced	class_weight= 'balanced'	Recall ≥ 0.82	Gestion déséquilibre	?

Remplir ce tableau AVANT de lancer les expériences = discipline scientifique

Markdown · arbitrage_modele.md

Note d'arbitrage – Choix modèle POC InduSense

Contexte

Prédiction pannes machines à partir des capteurs.
Dataset : 10 000 échantillons, 2 % de pannes.

Métrique principale

Recall panne (objectif ≥ 0.80)

Résultats expériences

Modèle	Recall	F1	Temps entraînement
Random Forest	0.83	0.74	2.3s
XGBoost	0.87	0.78	8.1s
LR balanced	0.76	0.69	0.4s

Modèle retenu

XGBoost – Recall = 0.87 (meilleur sur la métrique prioritaire)

Limites identifiées

- Validation sur un seul split (→ valider avec CV en S12)
- Hyperparamètres non optimisés
- Pas de test de stabilité temporelle

Reproductibilité — Seeds et versioning

aelion.fr

Un bon plan d'expérience est reproductible à l'identique par n'importe qui.

Python · setup reproductible

```
import numpy as np
import random
import os

# Fixer tous les seeds
SEED = 42
np.random.seed(SEED)
random.seed(SEED)
os.environ['PYTHONHASHSEED'] = str(SEED)

# Pour chaque estimateur sklearn/xgboost
rf = RandomForestClassifier(random_state=SEED)
xgb_model = XGBClassifier(random_state=SEED)

# Loguer la version des librairies
import sklearn, xgboost
print(f"sklearn={sklearn.__version__}")
print(f"xgboost={xgboost.__version__}")
```



random_state=42

Pour sklearn et XGBoost



np.random.seed(42)

Pour numpy



Versions librairies

sklearn, xgboost, pandas



Split identique

Même train/test pour tous les modèles



Code versionné

Commit Git avant chaque run